

Blood Glucose Forecasting Models - Usage Guide

This guide shows how to train and evaluate the RNN-based blood glucose forecasting models in this repository.

Available Scripts

1. `demo_bg_models.py` - Quick Demo with Synthetic Data

A self-contained demo that tests all models with synthetic blood glucose data.

Features:

- Generates realistic synthetic BG time series
- Tests RNN, LSTM, and GRU models
- Compares performance with clinical metrics
- No real data required - great for testing

Usage:

```
python3 demo_bg_models.py
```

What it does:

- Creates 2000 synthetic BG sequences (70-300 mg/dL range)
- Trains 3 models (RNN/LSTM/GRU) for 30 epochs each
- Evaluates with MAE, RMSE, MARD, Time-in-Range, Clarke Error Grid
- Shows comparison table and saves results

2. `run_bg_models.py` - Full Pipeline with Real Data

Comprehensive training script for the OhioT1DM dataset.

Features:

- Loads real OhioT1DM patient data
- Preprocesses and creates PyTorch datasets
- Trains models with proper validation
- Clinical evaluation with comprehensive metrics
- Saves trained models and results

Usage:

```
# Basic usage (patients 540 and 544, 50 epochs)
python3 run_bg_models.py
```

```
# Custom configuration
python3 run_bg_models.py --patients 540 544 552 567 --epochs 100 --
batch-size 64

# Quick test with fewer epochs
python3 run_bg_models.py --patients 540 --epochs 20 --sequence-length 12
--prediction-horizon 6

# Use only glucose features (unimodal)
python3 run_bg_models.py --patients 540 544 --unimodal --epochs 30
```

Available Arguments:

- **--data-dir**: Path to OhioT1DM data (default: `data/raw/ohiot1dm`)
- **--patients**: Patient IDs to use (default: 540, 544)
- **--epochs**: Training epochs (default: 50)
- **--sequence-length**: Input history length (default: 12 = 1 hour)
- **--prediction-horizon**: Future steps to predict (default: 6 = 30 min)
- **--batch-size**: Batch size (default: 32)
- **--unimodal**: Use only glucose (no insulin/carbs)
- **--version**: Dataset version (2018 or 2020)

Model Architecture

All models inherit from `BasePyTorchBGModel` and implement:

RNN Models Available:

1. Traditional RNN (`RNNBGModel`)

- Vanilla RNN with tanh/relu activation
- Good baseline for comparison

2. LSTM (`LSTMBGModel`)

- Long Short-Term Memory
- Handles long-term dependencies well
- Usually best for BG prediction

3. GRU (`GRUBGModel`)

- Gated Recurrent Unit
- Simpler than LSTM, often similar performance

Default Hyperparameters:

```
{
    'hidden_size': 64,      # Hidden layer size
    'num_layers': 2,        # Number of RNN layers
```

```

'dropout': 0.2,          # Dropout rate
'learning_rate': 0.001, # Adam learning rate
'batch_first': True      # PyTorch batch dimension
}

```

Input/Output Format

Input Sequences:

- **Shape:** (batch_size, sequence_length, features)
- **Sequence Length:** 12 timesteps (1 hour of 5-minute data)
- **Features:**
 - Unimodal: 1 (glucose only)
 - Multimodal: 4 (glucose, basal insulin, bolus insulin, carbs)

Output Predictions:

- **Single-step:** Next glucose value (30 min ahead)
- **Multi-step:** Next 6 glucose values (30 min horizon)

Evaluation Metrics

The models are evaluated using clinical blood glucose metrics:

Accuracy Metrics:

- **MAE:** Mean Absolute Error (mg/dL)
- **RMSE:** Root Mean Square Error (mg/dL)
- **MARD:** Mean Absolute Relative Difference (%)

Clinical Metrics:

- **Time in Range (TIR):** % time in 70-180 mg/dL
- **Time Below Range (TBR):** % time < 70 mg/dL
- **Time Above Range (TAR):** % time > 180 mg/dL

Error Grid Analysis:

- **Clarke Error Grid:** Clinical significance zones A-E
- **Parkes Error Grid:** Diabetes-specific error zones

Example Output

SUMMARY COMPARISON

Model	MAE	RMSE	MARD	TIR	Clarke A+B
RNN	18.45	24.32	14.2	78.5	92.1

LSTM	16.23	21.87	12.8	81.2	94.7
GRU	17.01	22.95	13.4	79.8	93.5

🏆 Best Model: LSTM (MAE: 16.23 mg/dL)

Clinical Interpretation

Good Performance:

- **MAE < 20 mg/dL**: Clinically acceptable
- **MARD < 15%**: Good accuracy for CGM standards
- **TIR > 70%**: Target for diabetes management
- **Clarke A+B > 95%**: Clinically acceptable predictions

Model Selection:

- **LSTM**: Usually best for BG forecasting
- **GRU**: Good alternative, faster training
- **RNN**: Simple baseline, may struggle with long sequences

Getting Started

1. **Quick Test**: Start with the demo

```
python3 demo_bg_models.py
```

2. **Real Data**: If you have OhioT1DM data

```
# Small test
python3 run_bg_models.py --patients 540 --epochs 20

# Full experiment
python3 run_bg_models.py --patients 540 544 552 567 584 596 --epochs 100
```

3. **Check Results**: Look in **results/** and **saved_models/** directories

Troubleshooting

Common Issues:

1. **Import Errors**: Make sure you're in the project root

```
cd /path/to/BG-forecasting
python3 demo_bg_models.py
```

2. **CUDA Out of Memory:** Reduce batch size

```
python3 run_bg_models.py --batch-size 16
```

3. **Training Too Slow:** Reduce epochs or patients

```
python3 run_bg_models.py --patients 540 --epochs 10
```

4. **Data Not Found:** Check data directory path

```
python3 run_bg_models.py --data-dir path/to/your/data
```

Next Steps

After running the basic experiments:

1. **Hyperparameter Tuning:** Modify [hyperparameters](#) in the scripts
2. **Model Comparison:** Try different architectures
3. **Feature Engineering:** Experiment with different input features
4. **Cross-Patient Validation:** Test models across different patients
5. **Clinical Validation:** Compare with existing CGM accuracy standards

For more advanced usage, see the individual model files in [benchmark/models/](#) and evaluation code in [benchmark/evaluation/](#).